

PLAIN-LANGUAGE RESEARCH PROPOSAL

DeepAlign-Bench | 完整人话版

方法、假设与实验不变，把每一步说清楚

适合组内共识 · 导师讨论 · 正式写作前校验

文档版本	v0.30 · 完整人话版
日期	2026 年 8 月 8 日
研究主线	为什么测 · 测什么 · 怎么测 · 如何判分 · 能声称什么

核心命题

同一任务和证据下，只改变用户；匹配结果要在两个用户方向都优于错配，并且相对 task-only 真的有帮助。

内容导航

1. 为什么现有评测不够
2. Case、Task 与 Persona 怎么构建
3. 实验如何运行
4. 最终交付物与过程怎么分工
5. Rubric、Metrics 与 Judge
6. 失败分类
7. 两个月执行计划
8. 顶会评审问题与论文主张边界
9. 参考文献

阅读方式：先看主图和研究概要；需要执行细节时看第 4-11 节；需要审稿防守时看第 12-13 节。

研究概要

一句话说明

我们要测的不是 Deep Research agent 能不能写出一份“看起来不错”的报告，而是：同一个研究任务交给不同用户时，它能不能根据用户真正相关的信息，交付不同但都正确的结果。

例如，两个人都问“我应该选哪一个硕士项目”。学校信息完全相同，但一个人预算紧、希望尽快就业，另一个人准备读博、重视科研训练。合格的 agent 不只是把两人的背景各写一遍，而应该改变比较标准、证据重点、风险提醒和最后建议。同时，学校事实、引用质量和基本分析不能因为“个性化”而变差。

我们真正要回答的五个问题

1. 给 agent 不同形式的用户信息，最后的交付物是否真的更适合这个用户？
2. 同一任务换一个同样合理的用户后，交付物是否在该变的地方变化、在不该变的地方保持？
3. agent 会不会忽略、误用、过度使用或泄露用户信息？
4. 在长任务、信息冲突、子 agent 交接或用户中途改主意时，个性化能力会不会丢失？
5. 我们设计的 rubric 和 judge 能不能稳定识别上述差异，而不是只学会偏爱某种写作风格？

两个月内要完成什么

主实验固定为 24 个 task family、每题两个强对比用户、4 种用户信息条件和 3 类 agent，最多 576 个 episode；其中 8 个 family 负责压力测试。另建 240-unit JudgeBench，并对至少 20% 输出做人评。领域专家负责事实和共同质量，目标用户负责判断 matched/swapped 哪份更适合自己；合成 persona 不能代替这一步。这个规模用于验证方法，不声称覆盖所有 Deep Research 模式；未测试、不适用和延期组合都会明确标出。

研究流程：从用户信息到可信的个性化评测

DeepAlign-Bench：什么才算“有效个性化”？

从 absolute fit 转向可识别的 cross-user specificity × real-user benefit



核心问题

固定 task、evidence、tools 与 budget，只改变目标用户：交付物是否既能“认对人”，又比 task-only 真正更有用，同时不牺牲共同质量与边界？

01 · GAP

现有 absolute fit 还缺什么

给定用户 U，报告 Y 好不好？

$PF_U(Y) = \text{高}$

可能只是通用高质量，不等于用户特异

1

通用答案也能高分

没有证明“换用户就必须换结果”

2

平均 CFA 可能正负抵消

一位用户受益，另一位可能被伤害

3

比 swapped 好 ≠ 真正有帮助

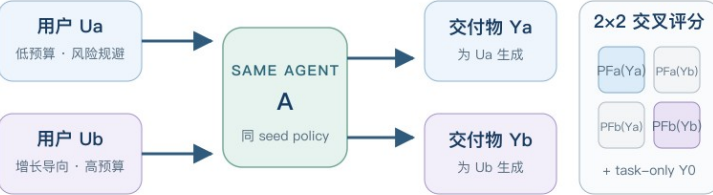
swapped 很差，也会制造虚假的优势

需要“特异性”和“受益”两个 estimand

02 · IDENTIFICATION

Controlled crossover：同一题，只交换用户

固定 T / E / tools / budget / output format



Specificity $\Delta a = PFa(Ya) - PFa(Yb)$ · $\Delta b = PFb(Yb) - PFb(Ya)$

CFA mean 展示平均效应；CFA min 防止跨用户方向补偿

Benefit $Ga = PFa(Ya) - PFa(Y0)$ · $Gb = PFb(Yb) - PFb(Y0)$

Gain mean/min 回答 matched 是否真的优于 task-only

03 · CLAIM GATES

四个门同时通过，才算有效

1

双向 specificity

$\Delta a > 0$ 且 $\Delta b > 0$ ；不是平均后看起来为正

2

双向 real benefit

$Ga \geq 0$ 且 $Gb \geq 0$ ；matched 不劣于 task-only

3

共同质量 no-harm

TQ / FR / must-hold 均过门；个性化不能换分

4

边界 no-violation

must-not / 隐私 / 权限 / 安全无 critical failure

Target-user blind match effect + family-level CI

论文贡献：一套可证伪的 personalization identification protocol，而不是另一个“更细”的总分

24 families · 48 user-task

4 signal conditions · 3 core agent tracks

8 anchors · S0-S3 stress

acquire · preserve · use · update

240-unit JudgeBench

target-user + expert gold · bias tests

主张边界

识别的是最终交付物的反事实特异性；不等于证明模型内部“真理解用户”

v0.30 · confirmatory unit = task family · family-blocked permutation / cluster bootstrap · no single compensatory leaderboard score

图1 方法与正式 Proposal 完全一致。主榜评最终交付物；轻量轨迹和受控压力分叉只用于解释保持、偏离与更新。

1. 为什么现有评测不够

1.1 研究问题是怎样一步步收敛的

第一步，Deep Research benchmark 先回答“报告一般好不好”：任务有没有完成、事实和引用是否可靠、分析是否完整。这是所有个性化结果都必须先过的底线，但它没有说明同一份报告更适合哪一个用户。

第二步，个性化研究开始回答“agent 是否理解了用户”。LaMP 用用户历史评测个性化生成，PersonaLens 在任务型对话里同时看偏好、任务成功和回复质量，PersonaMem 要求模型跟踪会变化的用户画像。[\[20\]\[22\]\[23\]](#) Setoka、PersonaTrail 和 APeB 又把信号扩展到异构记录、浏览轨迹和商品行为。[\[13\]\[15\]\[19\]](#) 所以“用户理解和历史利用没人测”已经说不通；这些工作大多停在响应选择、记忆问答或单域意图。

第三步，研究已经开始回答“理解之后有没有真的行动”。TravelPlanner+ 让用户模型改变旅行计划，ETAPP 和 ToolSpectrum 让画像与环境改变工具选择，Mem2ActBench 与 APOLLO 检查长期偏好能否落实到工具和参数。[\[21\]\[24\]\[25\]\[35\]\[36\]](#) TARS 还直接测了用户时间和认知负担。[\[16\]](#) 因此，“个性化行动无人评测”同样不是我们的 gap；这些任务多是单域计划或离散工具/GUI 动作，和开放式、多证据的 DR 最终交付物仍不同。

第四步，研究把时间和风险也纳入进来。RPEval 测无关记忆会不会导致不理性的个性化，PAHF 用澄清、记忆和反馈适应偏好变化，PerMemBench 问“哪些信息值得为这个用户写入”，Memora 与 CloneMem 测过期事实和多年数字轨迹。[\[30\]\[31\]\[32\]\[33\]\[34\]](#) PASB 和 PS-Bench 又说明，持久记忆不仅可能过期，还可能把迎合或危险意图长期合理化。[\[18\]\[39\]](#) 这要求我们的 irrelevant、stale、write、update 和 must-not 测试成为正式实验，而不是几个演示案例。

最后，PDR-Bench 与另一项 PDR 工作已经直接研究 persona 驱动的 Deep Research；[\[4\]\[27\]](#) MyScholarQA 还发现，合成用户和 LLM judge 会漏掉真人指出的九类细微个性化错误。[\[28\]](#) PDR-

Bench 会根据 task 和 persona 生成 P-Score 的权重与子标准，再评价目标、内容、呈现和可行性；它已经能表达“给定这个 task 和 persona，这份报告有多适合该用户”这一 absolute adaptation 问题。

DeepAlign 不重复回答这个问题，而是换一个 estimand：同一任务和证据换成另一个同样合理的用户后，两份报告是否各自更适合自己的用户？PDR-Bench 的 pairwise 比较回答“同一用户下哪个 agent 报告更好”；DeepAlign 的跨用户 2×2 matched/swapped 矩阵回答“只改变用户条件后，输出是否发生方向正确的变化”。这就是从 absolute adaptation evaluation 到 counterfactual personalization effect identification。

仅仅看到两份输出不同还不够。`must-change` 规定哪些决策必须随用户改变；`must-hold` 规定哪些事实、证据和共同质量不能改变；`must-not` 规定不能因为 persona 就额外推断、迎合或泄露什么。三者共同防止把随机差异、质量下降或过度个性化误认为有效 personalization。

但是 PDR-Bench 的自动评价不能被写成已经完全解决。它在 15 个 query、两个 agent 上做人类校准，最好的 GPT-5 judge 与人类 pairwise 顺序一致率只有 0.43，平均评分偏差为 1.40。[\[4\]](#) 另外，权重/子标准由一次 LLM 生成、分数再由另一个 LLM 生成，量尺本身可能波动；human panel 也不等于 persona 本人；事实分还依赖 claim 抽取、去重、网页抓取和支持判断；P/Q/R 平均又可能让事实失败被其他分补回来。正确说法是：PDR 的 rubric 已经能表示 absolute adaptation，但其 judge 校准和评分链条仍不足以直接支撑精细、跨系统的榜单。

还要避免另一个过度主张：matched/swapped 通过，也不能证明模型内部“真的理解了用户”。模型可能只是看到“不懂 AI”就调用短报告模板。为此我们要再检查：同一用户需求改写成结构化 persona、自然历史、澄清对话或去掉显眼关键词后，关键决策是否保持；只换无关人口属性或措辞时，不该变的事实和结论是否稳定。ACL 2026 的研究已经发现，同一 persona 换一种提示线索就可能改变测量结论；[\[40\]](#) PARL 也强调个性化 rubric 必须同时有代表性、用户一致性和区分力。

[41] DeepAlign 的 JudgeBench 同时补 PDR 尚未报告的 wrong-user swap、位置、长度、关键词和隐私诱饵，但 judge 改进与 estimand 创新要分开写。

1.2 我们采用反事实对照

一个任务 family 中，任务、证据、工具和预算保持不变，只改变用户。分别得到：

用户 U_a + 同一任务/证据 $\rightarrow$ 交付物 Y_a

用户 U_b + 同一任务/证据 $\rightarrow$ 交付物 Y_b

然后做交换评分： U_a 同时评价 Y_a 和 Y_b ， U_b 也同时评价 Y_b 和 Y_a 。只有 Y_a 更适合 U_a 、 Y_b 更适合 U_b ，同时共同事实和基本质量都过关，我们才说 agent 做到了个性化。

1.3 论文的核心贡献不应是“任务更多”

如果只是把 persona、任务和 agent 数量扩大，评审很容易把本项目看成 PDR-Bench、Setoka、PersonaTrail 或 APeB 的拼接版。[4][13][15][19] 因此核心贡献必须是可以单独验证的方法：

1. 用统一元数据描述不同 Deep Research 场景；
2. 用反事实任务族识别可观察的用户特异结果，而不是推断模型内部认知；
3. 用元数据自动选择适用 rubric，而不是一张表硬评所有任务；
4. 单独评估 judge 是否可靠；
5. 用压力测试区分获取、保持、利用和更新问题。

我们也要主动缩小首创表述：不声称首先研究 personalization、history、tool use、persistent state 或 temporal intervention，也不声称通过输出实验证明了模型“真正理解用户”。我们的候选贡献是把这些已有方向连接到广义 **Deep Research** 最终交付物的反事实特异性测量。如果 matched/swapped 人评不稳定，或同一用户需求换一种语义等价表达就改变结论，这个贡献就没有成立。

2. 一个测试样本到底包含什么

2.1 五组元数据

我们把一个 case 写成五组信息。这样做的目的不是增加术语，而是保证每次结果都能回答“在什么条件下、对什么任务、给了什么用户信息、测了什么系统”。

组别	用人话解释	主要字段
Research Task	用户到底要研究什么、拿结果做什么	使用情境、研究意图、领域、交付物、任务强度、风险
Research Environment	agent 在什么证据和工具条件下工作	frozen/live/private 证据、时效、搜索工具、预算、权限
User State	对当前任务真正有用的用户信息	目标、知识、约束、偏好、风险、受众、权限、动态状态
Signal Channel	用户信息以什么方式交给 agent	persona、对话历史、澄清、行为记录、工作区、反馈
Agent System	被测系统到底具备什么能力	模型版本、搜索、记忆、规划、多 agent、工具访问

这五组元数据组成 **Deep Research Evaluation Atlas**。Atlas 是“可测试空间的地图”，不是声称我们已经测完地图里的每一个格子。

2.2 四种行为测试

- **Acquire**：缺少关键信息时，agent 会不会提出有价值的澄清问题？
- **Preserve**：任务变长、信息变多或发生交接后，agent 能不能保留重要用户约束？
- **Use**：agent 知道用户信息后，是否真的把它落实到研究、判断和交付物？
- **Update**：用户状态按预注册事件发生变化后，agent 能不能采用当前真值并停止使用旧状态？

一个 case 等于：Atlas 中的一组坐标，加上一种行为测试，再加上事先写好的预期评价契约。

2.3 Coverage manifest

我们不会用“全面”这个词掩盖空白。每个组合都标记为：

- `tested`：进入主实验；
- `defined-only`：已经定义，但这版没有运行；
- `structurally-inapplicable`：这个组合本身不合理；
- `deferred`：合理，但因为两个月预算延后。

论文必须公开每个格子的样本量、模型数和运行条件。没有测试的格子不能用于支持结论。

3. 任务如何分类

3.1 不只分“博士题”和“日常题”

“PhD-level”和“daily”混合了用户身份、领域和任务难度。日常旅行规划可能需要大量搜索和复杂约束；博士也可能只要求核验一个事实。因此我们采用三个相互补充的维度。

3.2 使用情境

1. 个人与日常：旅行、消费、学习、职业、家庭计划；
2. 专业与企业：市场、采购、合规、技术和运营决策；
3. 学术与前沿：文献综述、prior art、研究设计和开放问题。

3.3 六类研究意图

1. 理解与综合；
2. 发现与枚举；
3. 比较与决策；
4. 评估与预测；

5. 规划、设计与排障；
6. 验证与审计。

3.4 任务强度

我们不把难度压成一个分数，而是分别标注：概念广度、逻辑层数、探索性、搜索分支数、信息时效、风险和可逆性、是否需要互动。

主数据用 18 个 family 覆盖“3 种使用情境 × 6 种研究意图”，再用 6 个 family 复测最重要或风险最高的单元，共 24 个 family。这个规模足以验证测量方法，但不足以声称每个细分类别都有稳定模型排名。

3.5 一个 task family 到底怎么造

先不要把很多论文里的 task/persona/rubric 直接拼在一起。建一张 source-to-design ledger：每篇论文的一条设计资产只能先标成 task seed、用户信息 construct、压力/失败假设、rubric/judge 方法或运行基础设施；同时写清采用什么、改了什么、为什么不采用其余部分。这样 PDR 的 persona-aware 评分、Agent-SafetyBench 的 failure 组织、PaperBench 的原子 rubric 和 EvalScope 的 adapter 不会被错误地当作同一层 taxonomy。

Task family 不是“旅行类”“学术类”这样的标签，而是一套能生成受控条件的蓝图。实际按八步做：

1. 从真实用户、专业工作流或已有 benchmark 收集一个确实需要多步研究的问题；
2. 固定所有用户共同的目标、证据截止时间、工具、预算和交付物；
3. 标注使用情境、研究意图、交付物和需求剖面；
4. 建 frozen evidence pack、困难负例、时间戳和权限视图；
5. 预先定义搜索 fan-out、冲突、上下文、handoff、更新等难度旋钮；
6. 配两个都自然、但会改变至少两项交付决策的用户；

7. 在模型运行前写 must-change / must-hold / must-not 和可接受替代；

8. 用参考 matched/swapped 输出做人评 pilot，分不出来就删题。

所以“同一个医疗 AI 采购研究 + 医院管理者/临床 AI 研究员”是一个 family：法规和产品事实不变，但 ROI/流程/合规与验证/漂移/复现的优先级必须变化。

3.6 难度怎么逐级增加

难度用六个独立旋钮表示：证据复杂度、用户信号复杂度、上下文长度、交接负载、权限敏感度、用户差异细微度。Anchor 内按同一前缀运行：S0 clean → S1 单个轻扰动 → S2 单个强扰动 → S3 两个正交扰动。Risk 说明失败伤害什么，failure mode 说明想暴露什么机制，stress level 说明触发有多强；三者不能混成一个总难度分。

4. Persona 和用户真值怎么做

4.1 Persona 不是人物小传

Persona 只是用户状态的一种展示形式。我们真正保存的是一个与任务相关的 user-state ledger。每条信息都记录：来源、时间、可靠度、敏感程度、与任务是否相关、是否允许用于推理、是否允许写进最终交付物。

同一份 ledger 可以被转成 structured persona、自然对话历史、澄清回答或 memory 记录。不同形式必须保持语义等价，否则测到的就不是“渠道差异”，而是“信息内容不同”。

具体不是让作者凭空写人物，而是：真实需求记录 → 提取目标/知识/硬约束/风险/受众/权限 → 先写 Ua/Ub 共享核心 → 只改 2-3 个有决策后果的字段 → 每条事实映射到一条评价契约 → 从同一 ledger 生成不同 signal view → 再生成 demographic-only、irrelevant、wrong-user、stale 和 redacted 负对照 → 原用户、相似用户与领域专家三方验证。没有 rubric 后果的 biography 细节从核心 persona 删除。

4.2 六道 persona-task 检查

1. 场景真实：这个用户自然地会提出这个任务；
2. 会影响结果：至少两条用户信息会改变内容、决策、深度、行动或披露边界；
3. 两个用户可区分：盲评者能说清楚两份正确交付物应该在哪里不同；
4. 仍有共同核心：事实、证据和基本质量要求不能随用户改变；
5. 信息最少且隐私可控：不加入完成任务不需要的个人信息；
6. 不靠刻板印象：关键偏好必须来自用户确认，不能由年龄、性别、职业等代理属性猜测。

4.3 用户信息来源

- `real_user_gold`：由真实用户确认的关键信息；
- `user_anchored`：从真实需求出发，经过隐私抽象后的主数据；
- `synthetic_control`：只用于错配、无关信息等负对照。

研究者自己推断但用户没有确认的偏好不能成为 gold。

4.4 每个 user-task 的真值包

- 所有用户都必须满足的共同要求；
- 目标用户必须满足的特异要求；
- 不得假设、不得披露或不得执行的事项；
- 可以接受的替代做法；
- 关键事实与证据；
- 严重错误对应的分数封顶；
- 应该澄清的问题以及可以不提问的替代路径；
- 两个用户的正确交付物应该在哪里不同、哪里相同。

这些内容必须在模型运行前冻结。LLM 可以帮助检查遗漏，但不能单独决定 gold。

人评也不能混成一个角色。领域专家或训练过的标注者检查事实、证据、共同质量和 must-hold；目标用户本人确认 must-change / must-not，并在不知道报告来自哪个条件时比较 matched 与 swapped。所有 real-user-gold family 和至少 8 个分层 family 都要有这种目标用户盲评；合成 persona 只用于可控压力测试和 judge 对抗题。MyScholarQA 已经说明，合成用户与 LLM judge 会漏掉真人真正介意的个性化错误。[\[28\]](#)

5. 实验如何运行

5.1 核心矩阵

```
24 个 task family
× 2 个强对比用户
× 4 种核心用户信息条件
× 3 类核心 agent
= 最多 576 个核心 episode
```

四种核心条件是：task-only、structured persona、语义等价自然历史、允许主动澄清。

三类核心 agent 是：M1 商业 Deep Research 产品、M2 在统一搜索和工具条件下运行的通用 harness、M3 可复现开源 DRA。M4 code agent、M5 multi-agent、M6 memory-enhanced 是架构 probe，只在适合的 anchor 上运行。

代码 agent、多 agent、记忆增强系统和第二个商业产品只在适合它们的 8 个 anchor family 中测试。不能为了让表格整齐，要求每个系统运行它根本不适合的任务。

5.2 三条评测轨道

- **E1 Frozen Harness**：只读证据快照、统一工具和预算；每次 reset 后以同一 seed 成对运行，形成因果主榜；
- **E2 Live Product/Web**：系统使用原生搜索和规划；记录日期、地区、订阅、版本、工具和 URL 快照，形成产品榜；

- **E3 Stateful Sandbox**：事件脚本在固定 checkpoint 注入澄清、冲突、handoff 和动态更新，共享前缀后分叉，形成压力与机制榜。

三条轨道是运行环境，不是 agent 类型。统一 adapter 至少要能 reset、提供 signal、运行到 checkpoint、注入 event、导出 artifact，并声明能保存 artifact-only、tool events、message events 还是 full state。三条轨道不能混成一个总榜。

不建议现在同时把三条都搭完整。先用 2 个 family、2 个 agent 把 E1 从 reset 跑到 2×2 评分；然后用 1 个 anchor 搭 E3 的 checkpoint、冲突和更新；最后拿 1 个商业产品做 E2 记录 smoke test。E1 和一个 E3 事件跑不通前，不要批量写几十个 task。

5.3 压力测试

这里要分清两件事：**persona 与 task 匹配**用于先做出一组有效的干净题；**压力测试**是在这组题上单独改一个输入或过程因素。Anchor family 只是从 24 个 family 中选出的 8 个合适实验宿主，不是 8 种 persona，也不等于 8 种失败。

先固定目标用户、任务、证据和预算，再做以下配对改变：

- 把用户 A 的 persona 错配给用户 B；
- 加入与任务无关的个人信息；
- 同时提供新旧冲突信息；
- 把关键信息埋进更长上下文；
- 在子 agent 交接时删除或保留用户模型；
- 用户中途更新目标。

8 个 anchor 固定覆盖：日常决策、学习/职业、金融信息、健康信息、企业采购/合规、软件生产、学术前沿、政策传播。前三个共同对照（clean、persona swap、无关信息）在所有 anchor 上运行；冲突、稀释、handoff 和动态更新用平衡不完全区组分配。要说“哪种因素跨任务更伤个性

化”，该主要 perturbation 至少要在 4 个适用 anchor 上做同前缀配对；只做 2 个只能叫探索性复现。

所以 anchor 真正回答的是：“固定任务、证据、预算和前缀后，long-context dilution / stale conflict / handoff / update 让 CFA、PF、invariance 或 MP 改变多少？”它不能仅凭“某类任务平均低”回答内部根因。只有轨迹可比时，才进一步把结果和 acquire/preserve/use/update 的过程证据联系起来。

每个扰动都要记录：什么保持不变、改了什么、何时插入、agent 当时能看见什么、配对 clean run 是哪一个，以及哪些结果应该变化/保持。对应指标分别是 ΔPF 、无关信息 invariance、冲突解析率、PF retention/AUC、handoff loss、update correctness 和旧状态残留率；同时检查事实、共同质量、长度和隐私是否受损。

6. 最终交付物和过程分别怎么评

6.1 最终交付物是主榜对象

论文的主要问题是“最后交付的报告、代码、表格或网页是否适合用户”，所以最终交付物可以作为主榜对象。它能回答：结果是否适合用户、matched 是否优于 swapped、个性化是否损害事实性、是否出现泄露或过度迎合。

6.2 只看最终结果不能解释所有机制

如果报告没有体现用户约束，仅看最后结果无法知道 agent 是从未读取、执行中忘记，还是记得但没有使用。因此：

- 全部样本保存必要的工具调用、检索、权限访问和交付物；
- 过程记录主要用于隐私和权限硬检查；

- 20%-30% 子集做受控压力分叉，用 memory ablation、handoff 和 dynamic update 定位保持与更新问题；
- 不做昂贵的逐句人工轨迹标注。

如果最后只完成 final-only 评测，论文必须把“获取、保持、利用、更新”的机制性结论降级，不能用最终输出反推内部原因。

7. Rubric 如何适配不同任务

7.1 不是所有任务共用一张表

现在有四个机器可读的 compiler contract，而不只是一句“根据元数据生成 rubric”：

- `case.schema.yaml`：这个 case 是什么任务、什么用户、什么环境、测什么 agent；
- `rubric_template_registry.yaml`：有哪些固定模板、什么元数据会激活它们；
- `rubric_leaf.schema.yaml`：每条最小评分项必须有哪些字段；
- `metric_binding.schema.yaml`：每条 leaf 进 TQ、FR、PF、MP 中的哪一个，派生指标再怎样计算。
- `rubric_module_library.yaml`：36 个预定义 module 的激活条件、leaf blueprint、binding、judge route 与适用范围；
- `data_factory.protocol.yaml`：从文献来源、task seed、user pair、contract 到 pilot 的造数顺序。

要讲准确：这些文件现在把接口、规则和贯通例子定清了，但自动校验和自动生成 bundle 的程序还没写完。第 1 周要实现 schema validator、模板路由、参数填充、leaf ID/hash、非法绑定拒绝和冻结导出；通过测试后才能叫“可执行 compiler”。

Compiler 的实际流程是：

```
冻结的 case metadata + user facts + 四类 contract + evidence/permission
→ 校验输入
```

- 按 intent / deliverable / operator / risk 选固定模板
- 把预算、截止时间、用户、证据等填进模板
- leaf expansion : 拆成可单独判断的原子项
- 检查覆盖、冲突、A/B 对称性、隐私和区分力
- 冻结 rubric bundle , 再运行所有 agent

每个 case 的模板来自六层：

共同质量
+ 个性化
+ 研究意图
+ 交付物类型
+ 行为测试
+ 风险模块
= 当前 case 的 rubric

所有 rubric 叶节点使用同一数据格式，但只有适用的模块才会被激活。例如 `compare_decide + decision_memo` 会选“比较方案、给出决策和 trade-off”的 intent 模板，以及“执行摘要、选择标准、风险边界、下一步”的 memo 模板；`code_and_docs` 则换成可运行测试、依赖和复现说明模板。领域事实只是填入模板的参数，医疗 claim 和市场 claim 仍使用同一种 evidence leaf schema，但由不同证据和专家阈值判定。

7.1.1 预定义 module library 长什么样

Family	数量	例子	什么时候用
Core	6	任务、事实、证据、推理、不确定性、可用性	所有 case 必选/条件选
Personalization	9	目标、内容、知识、约束、风险、工作流、受众、格式、动态状态	有授权 user fact + must-change 才激活
Intent	6	synthesis / discover / decide / assess / plan / audit	每个 case 主选 1 个
Deliverable	7	report / memo / table / code / slides / web / multi-file	每个 case 主选 1 个
Operator	4	acquire / preserve / use / update	受控诊断，不从 final-only

Family	数量	例子	什么时候用
			猜
Risk	4	隐私、安全、升级、冲突/过期	风险或 must-not 激活

我们的强处不是 Personalization 从 9 个继续扩到 15 个。模块越多，越容易 double count、调权重、让不同 case 分数不可比。真正的强处是：每个 personalization leaf 都必须有用户事实来源和 must-change；A/B 使用对称的模块形状；同一套 leaves 同时评分 matched/swapped；must-hold 和 must-not 阻止“变化越多越个性化”；最后还有目标用户和 JudgeBench 校准。这比“动态生成一张更细 rubric”更接近可证伪的测量协议。

7.2 Leaf expansion 到底是什么意思

它不是“compiler 先出一个分数，再把分数细化”，而是运行 agent 之前把一个大要求拆成多个能独立打分的小要求。

例如大要求是：

建议要符合 Ua 的预算和风险偏好。

它至少拆成：

- 1. U-A-BUDGET-01：第一阶段支出不超过 50 万；
- 2. U-A-PILOT-02：给出三个月可逆试点；
- 3. U-A-EXIT-03：给出可操作的继续与退出阈值。

每条 leaf 都要写清楚：属于哪个模板和 contract、适用于哪个用户、看交付物的什么证据、0/1/2 分分别是什么意思、权重、是否 hard gate、交给 verifier / judge / 用户 / 专家中的谁，以及直接进入哪个 metric。比如第 1 条可用数字 verifier + judge；0 分是超预算，1 分是提到预算但没有落实到方案，2 分是方案和阶段成本都满足。冻结后，任何 agent 都用同一标准。

7.3 四类评价契约

- `must-change`：不同用户必须改变的内容；
- `must-hold`：所有用户都必须保持的事实和质量；
- `must-not`：不得假设、泄露、越权或迎合的内容；
- `clarify-if-unknown`：缺少关键信息时应该提问或给条件分支。

7.4 三棵 rubric tree

1. **Common Task Quality**：任务完成、事实、证据、推理、行动性、文件完整性；
2. **User-Conditional Fit**：目标、内容、深度、约束、工作流、受众、动态状态；
3. **Misuse & Boundary**：刻板化、无关个性化、过期信息、隐私、越权和过度迎合。

7.5 Leaf 到分数怎么绑定

Leaf 类型	直接进入什么	说明
共同任务、 intent、deliverable	TQ；事实项还进 FR	所有用户都要做好
<code>must-change</code>	该用户的 PF	同一组 leaves 同时评 <code>matched</code> 和 <code>swapped</code>
<code>must-hold</code>	TQ + Neutral Invariance	单份看质量，跨两份看是否稳定
<code>must-not violation</code>	MP / hard gate	隐私或安全 critical 失败不能补偿
<code>clarify-if-unknown</code>	Clarification Correctness	擅自假设时另计 MP
<code>operator</code>	diagnostic delta	与同前缀 clean control 比，不混入基础总分

所以，TQ、FR、PF、MP 直接由 leaves 聚合；CFA 不直接绑定 leaf。Ua 的冻结 PF leaves 同时给 `Y_a` 和 `Y_b` 打分，得到 `PF_a(Y_a)`、`PF_a(Y_b)`；Ub 的 leaves 再得到另外两个值。CFA 是这四个 PF 的对角优势。要查某个分数来自哪里，只需沿 `criterion_id` → `direct_metric_bindings` → `aggregate` → `derived metric` 追踪。

7.6 Rubric 进入主实验前必须过七关

1. 每个关键元数据要么对应可判断的 rubric，要么明确只用于切片报告；
2. 人写的 matched 参考结果应明显优于 swapped 或错误版本；
3. 同一 user-state 换成语义等价 signal view 后，must-change / must-hold 判断应稳定；
4. 加入无关 persona、改变篇幅或文风不能提高不相关分数；
5. judge 在不同任务、交付物、用户信息渠道和风险等级上都要单独校准。
6. 检查不同 module 的语义重叠、leaf 数和权重敏感性，避免重复计分和量尺漂移；
7. 对 pilot 中现有 module 捕捉不到的错误做 open coding；只有同一残余 construct 在至少两个不同 family 重复出现，才新增 module。

无法稳定判断、不能区分 matched/swapped 的 rubric 必须删除或降级，不能靠调权重保留。

8. Metrics 怎么读

8.1 共同质量与事实性

- **TQ**：任务完成和基本可用性；
- **FR**：事实、证据、引用支持和来源质量。

如果共同质量或事实性不过门槛，不能靠个性化分补回来。

8.2 个性化表现

- **PF**：用户特异要求完成了多少；
- **MP**：误用、泄露、刻板化和过度迎合造成的惩罚；
- **PF - MP**：净个性化适配；
- **CFA**：匹配用户的报告相对错配报告有多少优势，但要把两位用户的方向分别保留下来。

```

 $\Delta a = U_a$  看  $Y_a$  相对  $Y_b$  的优势
 $\Delta b = U_b$  看  $Y_b$  相对  $Y_a$  的优势
 $CFA\_mean = (\Delta a + \Delta b) / 2$ 
 $CFA\_min = \min(\Delta a, \Delta b)$ 

```

CFA 的平均值大于 0 仍不够：如果 U_a 明显受益、 U_b 反而更适合错误版本，平均后也可能是正数。所以主结果必须同时给出 Δa 、 Δb 和 CFA_min ，只有两边都大于 0 才算“双向成立”。

还要回答另一件过去容易混掉的事：可区分不等于有帮助。主矩阵本来就有 task-only 报告 Y_0 ，因此再计算 $G_a = U_a$ 看 Y_a 相对 Y_0 的增益、 $G_b = U_b$ 看 Y_b 相对 Y_0 的增益。最后用二维图同时画“跨用户 specificity”和“相对 task-only benefit”：右上角才是真正有益的个性化；只在右下说明版本不同但没有给用户带来净收益。

最终成功条件不是一个新总分，而是四个门同时通过：两位用户的 matched 都优于 swapped；两位用户相对 task-only 都不变差；TQ/FR 与 must-hold 不下降；must-not、隐私和权限不违规。目标用户盲评再用 family 内配对检验确认 match effect，不能把同一个 family 的四格分数当四个独立样本。之后仍要报告语义等价 views 中最差的 CFA、Cue Gap、must-change/must-hold 一致率，以及无关 cue 是否改变结果。

8.3 长程与动态状态能力

- Retention：随着任务变长，用户适配保留了多少；
- Update correctness：状态变化后是否采用当前真值；
- Stale-state residue：旧状态仍进入最终交付物的比例；
- Pressure side effect：压力条件是否损害事实、任务质量或隐私。

8.4 榜单不能只给一个总分

主榜先检查 TQ、FR 和关键隐私门槛，再分四张 profile：clean 条件下的任务/交付物能力；不同 signal channel 的获取与利用；S0-S3 的风险/failure-mode 退化曲线；压力下的 must-not、隐私、权

限、弃权和共同质量副作用。每张 agent card 标注运行环境、trace level、覆盖率和 N/A；不同 evidence track、工具预算和 agent 类型不混排。

9. Judge 怎么做

9.1 四层评分

1. 确定性 verifier：文件、格式、预算、权限、代码测试；
2. 证据 verifier：claim 是否有支持、引用是否覆盖、来源是否可靠；
3. 强通用 judge：按照冻结的 rubric 逐项判断并定位证据；
4. 人类：目标用户判断是否有用，领域专家判断是否专业正确。

9.2 JudgeBench

用 240 个独立判分单元测试 judge，包括：正确答案、边界答案、位置交换、长度诱饵、漂亮格式诱饵、persona 关键词堆叠、隐私泄露和应当弃权的样本。

它还要专门覆盖 PDR 暴露或尚未覆盖的五类问题：低人类一致性、动态 criterion 重生成、通用 evaluator 与目标用户判断不一致、claim extraction/retrieval/support 链式误差、以及 P/Q/R 补偿关键失败。对应做 criterion 多次重生成稳定性、target-user matched/swapped、claim-recall 审计、抓取失败单列和 hard-gate 对照。

主 judge 至少要达到预注册的一致性、位置稳定性、群体差距和校准门槛。不达标的 rubric 改成人评或不进入主榜。

9.3 为什么 SFT scorer 不是当前主线

“人工 0/1 + GPT 理由”可以训练便宜 scorer，但 GPT 理由不是新的 ground truth，它可能只是在已知标签后合理化。两个月主线采用：

确定性/证据 verifier → 强通用 judge → 20% 分层人评和分歧仲裁

只有第 4 周前已经有 240 个高质量样本且主实验不受影响时，才做 SFT scorer 附录。它必须按 task family 和 agent 隔离测试；如果不能跨 family 泛化，只能做高置信样本分流。

10. 我们要报告哪些失败

- 1. 缺信息时既不获取也不澄清；
- 2. 没有依据就猜用户属性；
- 3. 检索错了、漏了或拿到无关用户信息；
- 4. 明明知道预算、权限或格式要求却忽略；
- 5. 新旧信息冲突时仍使用过期内容；
- 6. 会复述用户信息，但没有落实到结果；
- 7. 在不该变化的地方强行个性化；
- 8. 越权访问或披露敏感信息；
- 9. 长任务或 agent 交接后丢失约束；

这些是 case 的“预期测试目标”，不是运行结果。真实失败必须由输出或轨迹证据另行标注，并保留 other/emergent 类，避免只发现作者预设的错误。

11. 两个月执行计划

周	必须完成	如果失败怎么办
1	冻结 Atlas、schema、coverage manifest 和 24 个 family 配额	缩小 ontology，不增加新分支
2	24 个 family 草案、48 个 user state、persona 兼容性检查	删除没有稳定用户差异的任务

周	必须完成	如果失败怎么办
3	真值包、四类契约、rubric modules 和小样本人评	不能区分 matched/swapped 的 rubric 不进主榜
4	240-unit JudgeBench、judge 基线、6 个 family dry run	judge 不过门就缩成更多人评
5	三类核心 agent 跑完主矩阵	冻结版本，停止新增 agent
6	anchor 压测、20% 人评、错误 open coding	只保留证据充分的机制结论
7	统计、bootstrap、覆盖审计和论文主要表格	删除不受数据支持的支线
8	复现、结果冻结、全文整合和匿名发布材料	不新增 taxonomy、agent 或任务类型

12. 顶会评审最可能问什么

12.1 这是不是 PDR-Bench 扩大版

应先承认 PDR-Bench 已经测 task-persona 条件下的 absolute adaptation；再证明 DeepAlign 的跨用户 2×2 对角优势和预冻结契约能识别 counterfactual personalization effect。同时可以公平指出 PDR 的最佳 PCA=0.43、15-query/2-agent 校准、动态 criterion variance、非 target-user validity 和复合事实核验链等可靠性边界。前者是方法创新，后者是测量改进，不能混成一个 claim。

12.2 Persona 是不是作者想象

主数据必须来自真实用户确认或 user-anchored 需求。人口属性不能直接生成偏好。matched 交付物需要目标用户盲评，persona-task pairing 必须通过六道检查。

12.3 元数据很多，但真正测得很少

明确区分 ontology scope 和 empirical coverage；公开 coverage manifest、选择规则和缺口。没有测试的组合不做结论。

12.4 不同任务的 rubric 能不能比较

统一的是 rubric schema、评价契约和校准程序，不假设所有模块天然等距。论文主要报告任务内反事实效应、模块完成率和分层结果；没有共同 anchor 就不制造统一总榜。

12.5 Final-only 能不能证明获取、保持、利用和更新

不能。最终交付物支持“结果是否适合用户”的主张；机制性主张需要受控重跑或轨迹证据。若诊断实验没有完成，就缩小论文主张。

12.6 Judge 会不会循环定义答案

Rubric 在输出前冻结；客观项优先用 verifier；judge 只做需要语义判断的叶节点；独立 JudgeBench 和人评决定哪些指标可以进入主榜。

13. 论文最后可以声称什么

如果主要门槛通过，论文可以声称：

- 提供了一套机器可读的个性化 Deep Research 评测空间；
- 反事实任务族能识别交付物的用户特异性，语义等价信号测试再检查其是否只依赖表面 persona 线索；
- 不同用户信息渠道和 agent 架构会产生可诊断差异；
- 模块化 rubric 与独立 judge 校准可以支持可复现评测；
- 若完成诊断子集，可以报告保持和更新的受控压力证据。

论文不能声称：已经穷尽所有 Deep Research 模式；每个细分任务都有稳定排名；一个总分可以跨 frozen/live、商业/受控系统和所有交付物直接比较；最终输出能够完整解释 agent 内部何时偏离。

参考文献

- [1] OpenCompass Team. OpenCompass: A Universal Evaluation Platform for Large Language Models. arXiv:2605.19276, 2026.
- [2] ModelScope. EvalScope Introduction. https://evalscope.readthedocs.io/en/refact_readme/get_started/introduction.html
- [3] Zhang et al. Agent-SafetyBench: Evaluating the Safety of LLM Agents. arXiv:2412.14470.
- [4] Towards Personalized Deep Research: Benchmarks and Evaluations. arXiv:2509.25106.
- [5] Wang et al. LiveResearchBench: A Live Benchmark for User-Centric Deep Research in the Wild. arXiv:2510.14240.
- [6] Sharma et al. ResearchRubrics: A Benchmark of Prompts and Rubrics for Evaluating Deep Research Agents. arXiv:2511.07685.
- [7] Java et al. Characterizing Deep Research: A Benchmark and Formal Definition. arXiv:2508.04183.
- [8] Yoran et al. AssistantBench: Can Web Agents Solve Realistic and Time-Consuming Tasks? arXiv:2407.15711.
- [9] Liang et al. Holistic Evaluation of Language Models. TMLR, 2023.
- [10] Ribeiro et al. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. ACL, 2020.
- [11] Reuel et al. BetterBench: Assessing AI Benchmarks, Uncovering Issues, and Establishing Best Practices. arXiv:2411.12990.
- [12] Sokol et al. BenchmarkCards: Standardized Documentation for Large Language Model Benchmarks. NeurIPS Datasets and Benchmarks, 2025.
- [13] Zeng et al. Setoka. arXiv:2607.27056, 2026.
- [14] Qian et al. Toward User-Conditioned Evaluation of Personal LLM Agents under Temporal Interventions. arXiv:2607.21635, 2026.
- [15] Yang et al. PersonaTrail. arXiv:2607.20482, 2026.
- [16] Todisco et al. TARS. arXiv:2607.15948, 2026.
- [17] Yang. Self-Aware Recursively Self-Improving Agents for Personal Singularity. arXiv:2607.12254, 2026.
- [18] Mao et al. Agents Don't Just Agree, They Remember. arXiv:2607.10526, 2026.
- [19] Yang et al. APeB. arXiv:2607.03162, 2026.
- [20] Salemi et al. LaMP: When Large Language Models Meet Personalization. ACL, 2024. <https://aclanthology.org/2024.acl-long.399/>
- [21] Singh et al. Personal Large Language Model Agents: A Case Study on Tailored Travel Planning. EMNLP Industry Track, 2024. <https://aclanthology.org/2024.emnlp-industry.37/>
- [22] Zhao et al. PersonalLens. Findings of ACL, 2025. <https://aclanthology.org/2025.findings-acl.927/>
- [23] Jiang et al. Know Me, Respond to Me. arXiv:2504.14225, 2025.
- [24] Hao et al. Evaluating Personalized Tool-Augmented LLMs from the Perspectives of Personalization and Proactivity. ACL, 2025. <https://aclanthology.org/2025.acl-long.1064/>
- [25] Cheng et al. ToolSpectrum. arXiv:2505.13176, 2025.
- [26] Zhang et al. PRIME. EMNLP, 2025. <https://aclanthology.org/2025.emnlp-main.1711/>
- [27] Li et al. Personalized Deep Research: A User-Centric Framework, Dataset, and Hybrid Evaluation. arXiv:2605.10530, 2026.
- [28] Balepur et al. Language Models Don't Know What You Want. ACL, 2026. <https://aclanthology.org/2026.acl-long.723/>
- [29] Garbacea et al. Personalized Benchmarking: Evaluating LLMs by Individual Preferences. Findings of ACL, 2026. <https://aclanthology.org/2026.findings-acl.31/>
- [30] Feng et al. How Does Personalized Memory Shape LLM Behavior? arXiv:2601.16621, 2026.
- [31] Liang et al. Learning Personalized Agents from Human Feedback. arXiv:2602.16173, 2026.
- [32] In et al. Personalize-then-Store. arXiv:2605.25535, 2026.
- [33] Uddin et al. From Recall to Forgetting. Findings of ACL, 2026. <https://aclanthology.org/2026.findings-acl.1337/>
- [34] Hu et al. CloneMem. ACL, 2026. <https://aclanthology.org/2026.acl-long.1549/>
- [35] Shen et al. Mem2ActBench. ACL, 2026. <https://aclanthology.org/2026.acl-long.370/>
- [36] Chen et al. Towards Preference Following in Tool Calling Language Agents. Findings of ACL, 2026. <https://aclanthology.org/2026.findings-acl.1676/>
- [37] Lyu et al. PersonalAlign. ACL, 2026. <https://aclanthology.org/2026.acl-long.1669/>
- [38] Wang et al. OPeRA. ACL, 2026. <https://aclanthology.org/2026.acl-long.2033/>
- [39] Guo et al. When Personalization Legitimizes Risks. ACL, 2026. <https://aclanthology.org/2026.acl-long.1260/>
- [40] Weeber et al. One Persona, Many Cues, Different Results. ACL, 2026. <https://aclanthology.org/2026.acl-long.2079/>
- [41] Qiu et al. Preference-Aware Rubric Learning for Personalized Evaluation. arXiv:2605.31545, 2026. <https://arxiv.org/abs/2605.31545>